



1

10

1650 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

لماذا Assembly ؟

أ.عبير ميا



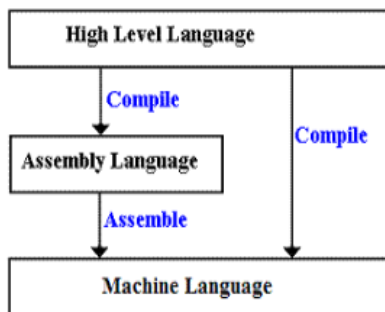
RB Informatcs; 31/03/2026

بنیان الحاسوب ا

مستويات البرمجة:

- منذ تعلمنا البرمجة، نكتب الكود باللغة التي نختارها (C++, Java, ...) وهي لغات عالية المستوى ويعمل الكود (غالبا 😊)
- اللغات عالية المستوى مثل JavaScript, Java ... تتميز بكونها لغات سهلة الفهم بالنسبة للمبرمج لكنها غير مفهومة بالنسبة للمعالج الذي يتعامل فقط بلغة الآلة Machine Language. هنا، يأتي دور المترجم Compiler الذي يحول اللغات عالية المستوى إلى لغة الآلة.
- ولكن ماذا يحدث؟؟

حسب لغة البرمجة:



- إما يقوم الـ compiler بترجمة الكود إلى لغة الآلة (machine language).
- أو يقوم الـ compiler بترجمة الكود إلى لغة الـ assembly
- ويقوم الـ assembler بتحويل كود الـ assembly إلى لغة الآلة.

لماذا نحرس البرمجة بلغة التجميع Assembly

- تتيح للمبرمج فهم كيفية عمل المعالج وإدارته لموارد الحاسوب مما يطور مهارات المبرمج باستخدام الـ High Level Languages ويؤدي إلى كتابة برامج ذات كفاءة أكبر
- تستخدم لبرمجة الأنظمة المدمجة وأنظمة الزمن الحقيقي (Embedded/Real Time systems)، مثل: البرادات الحديثة والغسالات والسيارات الحديثة، لأنها تمنح المبرمج تحكماً مطلقاً في موارد النظام.
- يمكن استخدام لغة التجميع Assembly language لبرمجة أجزاء من الكود يصعب على اللغات عالية المستوى تنفيذها أو لتنفيذها بشكل محسن optimized.
- تحسين كفاءة برمجة اللغات عالية المستوى High-Level Optimization، من خلال فهم كيفية التعامل مع الذاكرة وعمل المكس Stack و الكومة Heap.
- الأمن السيبراني والهندسة العكسية (Cybersecurity & Reverse Engineering)، من خلال: تحليل البرمجيات الخبيثة Malware Analysis

اختبار الاختراق واستغلال الثغرات: فهم كيفية التلاعب بـ "مؤشر التعليمات" (Instruction Pointer) لتوجيه البرنامج لتنفيذ كود ضار أو كسر حمايته (Cracking) وإيجاد serial وغيرها....
والكثير من التطبيقات الأخرى كتطوير نظم التشغيل OS، وبناء المترجمات Compilers و...و...

البرمجة بلغة التجميع

لو كنا نكتب كود بلغة ++C مثل $int\ z = x + y$; وهو كود لجمع عددين معرفين سابقا فحسب معمارية المعالج ونوعه تختلف كتابة الكود:

MIPS
(RISC)

```
lw $t0, 0($s1) # تحميل من الذاكرة x
lw $t1, 0($s2) # تحميل من الذاكرة y
add $s0, $t0, $t1 # z = x + y
```

Intel x86
(CISC)

```
mov eax, [ebx] # نقل x للسجل
add eax, [ecx] # جمع x مع الذاكرة
```

RISC-V
(RISC)

```
lw x5, 0(x10) # تحميل من الذاكرة x
lw x6, 0(x11) # تحميل من الذاكرة y
add x7, x5, x6 # z = x + y
```

في هذا المقرر سنتعامل مع RISC (Reduced Instruction Set Computing) في معالج MIPS R3000 حيث يقوم برنامج MARS بعمل محاكي للمعالج و assembler.

ما هو معالج MIPS R3000؟

معالج يحتوي على 32 سجل (register) كل سجل منها مؤلف من 32 bit (32 قلاب كل منها يحفظ بت واحد).

السجلات:

مرقمة من 0 إلى 31 ((لها أسماء))

(لاحقا في دراستنا سنعرف لماذا هذه الأسماء وأن السجلات تستخدم لغرض معين حسب نوعها)

يمكن الإشارة إلى أي سجل إما برقمه أو اسمه مسبقا بإشارة \$، مثال: \$3, \$t1, ...

يتم استخدام السجلات بدل المتحولات، أي أأزن ما أأناأ في السجلات وأقوم بتمرير السجلات ك parameters للتعليمات والدوال للقيام بعمليات عليها.

مأأوى السجل \$zero(\$0) دوما 0

يمكن وضع comments بأستأأام #

#	Name
0	zero
1	at
2, 3	v0, v1
4..7	a0..a3
8..15	t0..t7
16..23	s0..s7
24, 25	t8, t9
26, 27	k0, k1
28	gp
29, 30	sp, fp
31	ra

■ مثال: تعليمة add

بفرض لدينا السجلات التالية وفيها قيم ابتدائية:

$\$8 = 10, \$9 = 20, \$10 = 40$

فإن التعليمة: $add\ \$10, \$8, \$9$ تقوم بجمع مأأوى السجلين 8 و 9 وتضع الناتأ في السجل 10 فيصأ مأأوى السجل 10 يساوي 30 بدل 40.

حيث: add \$10, \$8, \$9

تسمى add بالعملية operation و \$10, \$8, \$9 بالمعاملات operands وتقسم إلى:

Destination operand: \$10

Source operands: \$8, \$9

ما يحدث (بشكل مبسط) أن محتوى السجلين 8 و 9 ينقل إلى وحدة الحساب والمنطق في المعالج (ALU) ثم يوضع الناتج في السجل 10.

مثال: تعليمة addi

ماذا لو أردنا جمع محتوى سجل مع عدد ليس مخزنًا ضمن سجل آخر؟

هنا نستخدم تعليمة addi وهي:

add immediate : (وهو عنصر حالي غير مخزن في أي سجل)

بفرض محتوى السجل 4 = \$9

addi \$2, \$9, 5

فإن:

تجعل محتوى السجل 2 يساوي 9

ملاحظة: تعليمة addi تأخذ عدد واحد فقط وفي النهاية أي:

addi register, register, number

يمكن استخدام أعداد سالبة، مثال: لدينا 6 = \$8 وقمنا بـ:

addi \$4, \$8, -2

فإن 4 = \$4

ملاحظة: يمكن جمع العدد في تعليمة addi بالتمثيل الست عشري (hexadecimal) بالشكل:

0x012

حيث 0x للدلالة على أن ما بعدها عدد ممثل بالنظام السداسي عشر.

مثال:

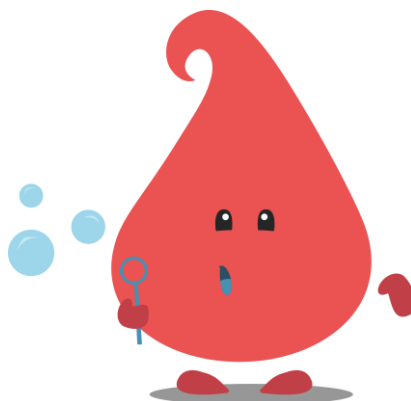
addi \$8, \$0, 0x16

تجعل محتوى السجل 8 يساوي 22 لأن 0 = \$0 دائما و 16 بالنظام السداسي عشر تساوي 22:

$6 \times 16^0 + 1 \times 16^1$

ملاحظة: يمكن استخدام addi register, \$0, number لتخزين العدد في السجل الذي نريد حيث أن الجمع

مع 0 لا يؤثر على العدد.



تدريب: بفرض أن المحتوى الابتدائي لكل سجل هو رقم السجل +10 مثال (3 = \$3)، ما هو محتوى سجل الوجهة بعد تنفيذ تسلسل التعليمات التالية (بالتمثيل العشري):

- 1) sub \$3,\$2,\$3 # \$3 = 12 - 13 = -1
- 2) addi \$4,\$4,-5 # \$4 = 14 - 5 = 9
- 3) andi \$8,\$5,0xC # \$8 = 15 and 12 = 0..01111 and 0..01100 = 0..01100 = (12)₁₀
- 4) or \$6,\$7,\$4 # \$6 = 17 or 9 = 0...010001 or 0..01001 = 0..011001 = (25)₁₀
- 5) andi \$12,\$5,1 # \$12 = 15 and 1 = 1111 and 1 = 1

✧ يمكن الاستفادة من القيام ب and لعدد مع ال1 في معرفة اذا كان العدد فردي أم زوجي، حيث ال1 بالتمثيل الثنائي هي:

000000000000001 (أصفار بحجم التخزين) فيكون الناتج إما 0 أو 1 في أول بت (LSB) لأن باقي تمثيل ال1 هو أصفار و and number = 0 فإذا كان العدد زوجي يكون جواب العملية 0 وإذا كان فردي يكون جواب العملية 1 لاحظ أن التمثيل الثنائي لعدد يكون على الشكل 1 2 4 8 16 32 64 (مضاعفات الاثنين) و ليكون العدد فردي ف أنه يحتاج لل 1 (التي هي أول بت من اليمين أو ما يسمى بال LSB) و بما أن التمثيل الثنائي للواحد هو واحد من اليمين و على يساره اصفار فإذا كان العدد فردياً تكون الخانة الأيمن فيه تساوي 1 و عندها يكون 1 (and) 1 = 1 و باقي الخانات تساوي صفر لأن ال 0 (and) 0 = 0 اي عدد 0 = 0 بينما اذا كان زوجيا ف تكون الخانة الأيمن تساوي صفر و عندها يكون 0 (and) 0 = 0

- 6) ori \$10,\$8,3 # \$10 = 0..01100 or 11 = 0..01111 = 15
- 7) sub \$13, \$8,\$10 # \$13 = 12 - 15 = -3

من المهم: القدرة على التحويل بين أنظمة العد بشكل سريع، نترك لكم جدول للتذكير:

Binary	Hex	Decimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

2 ¹ =	2	10 ₂
2 ² =	4	100 ₂
2 ³ =	8	1000 ₂
2 ⁴ =	16	10000 ₂
2 ⁵ =	32	100000 ₂
2 ⁶ =	64	1000000 ₂
2 ⁷ =	128	10000000 ₂
2 ⁸ =	256	100000000 ₂
2 ⁹ =	512	1000000000 ₂
2 ¹⁰ =	1,024	10000000000 ₂

في النظام العشري عندما نضرب بقوى ال 10 نضيف أصفار إلى نهاية العدد بمقدار القوة، كذلك في الثنائي عندما نضرب بقوى ال 2 نضيف أصفار إلى يمين التمثيل الثنائي بمقدار القوة، أمثلة:

$$30_{10} = 15 \times 2 = 1111_2 \times 2^1 = 11110_2$$

$$40_{10} = 10 \times 4 = 1010_2 \times 2^2 = 101000_2$$

$$63_{10} = 64 - 1 = 111111_2$$

$$1025_{10} = 10000000001_2$$

$$0x1F + 0x1 = 0x20$$

$$0x1008 + 0x4 = 0x100C$$

أنماط التعليمات MIPS instruction formats:

تتألف التعليمات في MIPS من 32 bit ويوجد منها 3 أنماط:

R-type، ولها op = 0، على الشكل: operation, rd, rs, rt، مثل تعليمات: add, sub, and, or, xor...

op	rs	rt	rd	shift	func
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-type ولها (op ≠ 0, 2, 3)، على الشكل: operation, rt, rs, immediate

مثل تعليمات: addi, andi, ori, lw, sw, beq, bne

هامش: تعليمات لها علاقة بالذاكرة، تعليمات لها علاقة بال branch (لاحقاً)

op	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

J-type، على الشكل: J target address، ولها:

Label J (op = 2)

Jal Func (op = 3)

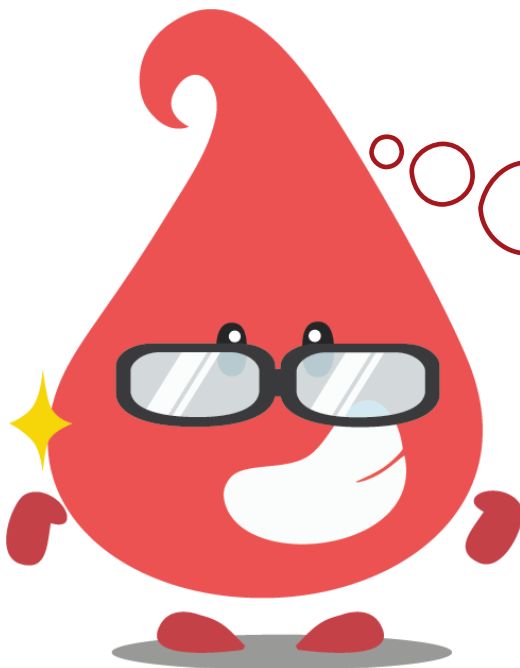
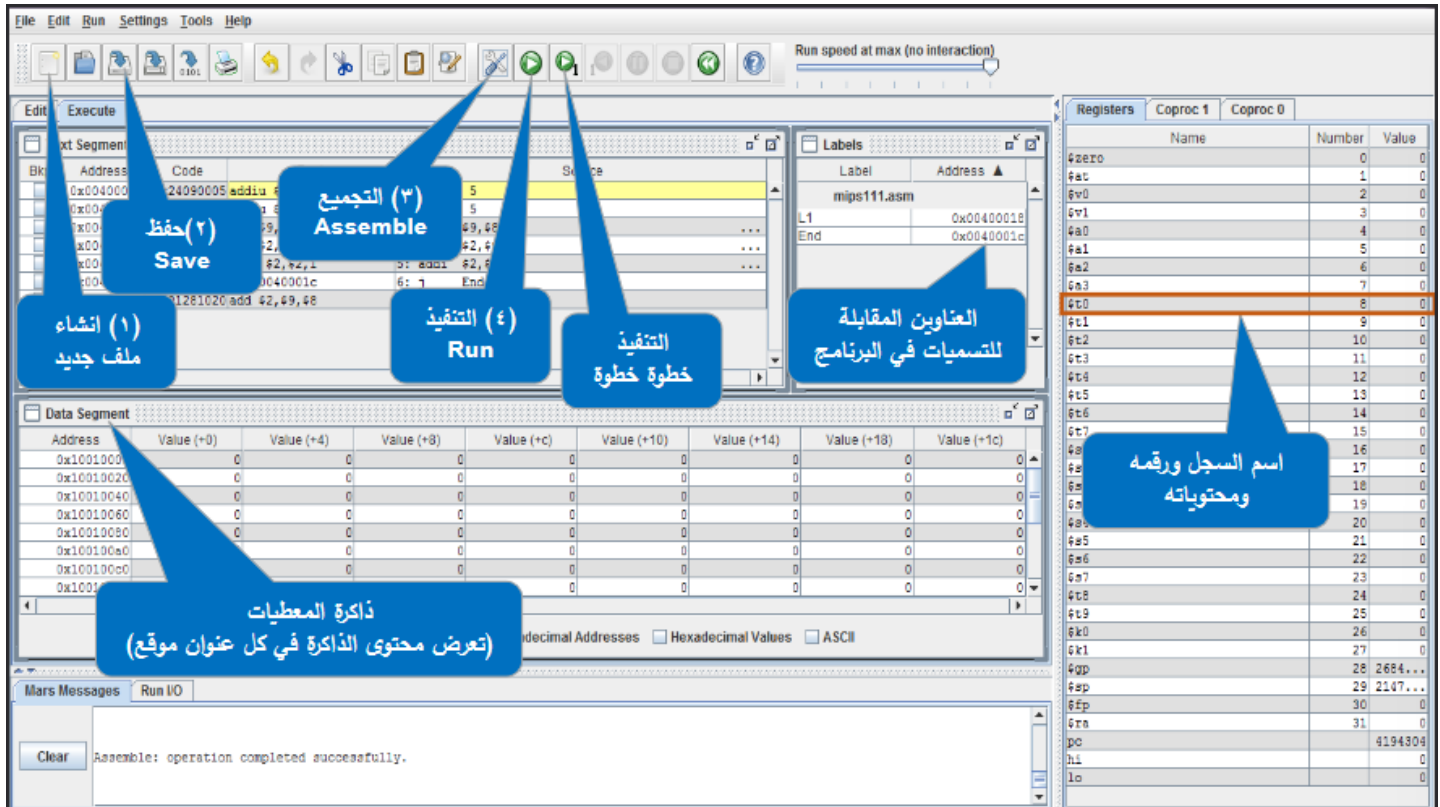
op	immediate (target address)
6 bits	26 bits

ملاحظات:

- لاحظ أن ترتيب تخزين التعليمات في الأنواع الثلاث قد يختلف عن ترتيب كتابتها
- يمكن معرفة نوع التعليمات من ال 6 bits الخاصة ب op، فمثلاً التعليمات: R-type من ملاحظة أول 6 بتات فإنها R-type 000000010110011101010101010101

برنامج MARS

هو برنامج محاكاة لتنفيذ البرامج المكتوبة بلغة التجميع MIPS، وهو اختصار للعبارة MIPS Assembler and Runtime Simulator، فهو إذاً مجمع وبنفس الوقت برنامج محاكاة لمعالج MIPS أثناء التنفيذ، البرنامج مكتوب بلغة جافا ويتم توزيعه على شكل ملف jar. قابل للتنفيذ.



Push yourself, because
no one else is going to
do it for you

أسئلة لمحاضرة العملي الأولى

1- تمر جميع لغات البرمجة عالية المستوى بنفس خطوات الترجمة إلى لغة الآلة (صح أم خطأ)

الإجابة: خطأ

2- ماهي حالة الاستخدام المناسبة للغة التجميع؟

a- برمجة الأنظمة المدمجة / أنظمة الزمن الحقيقي

b- بناء تطبيقات الويب

c- الهندسة العكسية

d- a & c

الإجابة: d

3- نستفيد من لغة التجميع من خلال:

a- برمجة أجزاء معينة من التعليمات البرمجية التي لا تستطيع لغات البرمجة عالية المستوى تنفيذها

b- فهم كيفية التعامل مع الذاكرة

c- فهم كيف يعمل ال Stack وال Heap (المكدس والكومة)

d- جميع ما سبق

الإجابة : d

4- كم عدد المسجلات التي يمتلكها معالج MIPS R3000؟

a- 32

b- 16

c- 8

d- 64

الإجابة: a

5- كم عدد ال bits (الخانات) في كل مسجل، وكيف تُخزن هذه الخانات؟

a- 32 بت ، باستخدام قلابات

b- 64 بت ، باستخدام قلابات

c- 32 بت ، باستخدام أسلاك

d- 64 بت ، باستخدام أسلاك

الإجابة : a

6- العلامة التي تسبق اسم أو رقم السجل:

a- \$

b- @

c- #

d- لا شيء مما سبق

الإجابة : a

7- أي تعليمة تأخذ قيمة مسجل وتضيفها إلى عدد معطى وتخزنها في مسجل آخر؟

add -a

addi -b

sub -c

subi -d

الإجابة: b

8- أي تعليمة تحتوي على ثلاثة معاملات جميعها مسجلات ؟

addi -a

subi -b

andi -c

d- لا شيء مما سبق

الإجابة: d

9- التعليمة التي تُجري عملية منطقية على المسجلين v0 و v1 وتخزن النتيجة في المسجل a0 هي:

add \$a0 , \$v1 , \$v0 -a

and \$v0 , \$ v1 , \$ a0 -b

or \$a0 , \$v0 , \$v1 -c

xor \$v1 , \$a0 , \$v0 -d

الإجابة : c

10- إحدى طرق تعيين قيمة ابتدائية لمسجل (لنفترض a0) هي:

add \$a0 , \$t0 , \$t1 -a

addi \$a0 , \$v0 , 0x16 -b

or \$a0 , \$0 , \$7 -c

ori \$a0 , \$0 , 130 -d

الإجابة : d

11- تقسم البتات في التعليمات من النوع R-type إلى :

a- 6 أقسام

b- 4 أقسام

c- قسمان

d- لا شيء مما سبق

الإجابة : a

YOU ARE CHOSEN

12- إذا كانت أول ستة بتات من تعليمة (op) تساوي صفراً ، فإن نوع التعليمة هو :

R-type -a

I-type -b

J-type -c

N-type -d

الإجابة : a

13- نوع التعليمة if op=5 :

R-type -a

I-type -b

J-type -c

N-type -d

الإجابة : b

14- التعليمة "beq" هي من النوع :

R-type - a

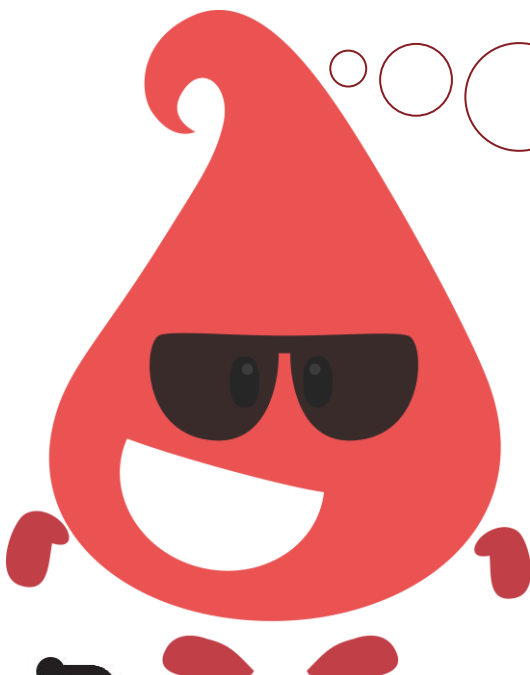
I-type -b

J-type -c

N-type -d

الإجابة : b

If your dreams don't scare
the weak-minded, then
your vision is still too
small



تتبع الكود التالي:

لنفترض أن قيمة كل سجل هي رقم ذلك السجل مضروباً بـ 2 ، مثلاً قيمة \$3 هي 6:

```
sub $3 , $2 , $3
addi $4 , $4 , -5
andi $8 , $5 , 0xC
or $6 , $7 , $4
andi $12 , $5 , 1
ori $10 , $8 , 3
sub $13 , $8 , $10
```

ماهي قيم كل سجل من السجلات التالية؟

13\$, 12\$, 10\$, 8\$, 6\$, 4\$, 3\$

الإجابة :

\$3=-2 \$4=3 \$6=15 \$8=8 \$10=11 \$12=0 \$13=-3

لنفترض أن :

a: \$t0 , b: \$t1 , c: \$t2 , d: \$t3

w: \$v0 , x: \$v1 , y: \$s0 , z: \$s1

1- قم بتعيين أي قيم ابتدائية ترغب بها للمتغيرات a و b و c و d باستخدام أي تعليمة مناسبة ، بشرط أن تكون قيمة d سالبة.

2- اكتب كود التجميع الذي ينفذ العمليات التالية :

$w = 3d$

$x = a + b - d + 1$

$y = b \text{ and } c$

$z = (a \text{ or } b) \text{ and } c$

الإجابة : (هذا أحد الحلول ، وليس الحل الوحيد):

#setting the initial values

addi \$t0 , \$0 , 4

addi \$t1 , \$0 , 12

ori \$t2 , \$0 , 7

ori \$t3 , \$0 , -4

#w = 3*d = -12

mul \$v0 , \$t3 , 3

#x = a + b - d + 1 = 21

add \$v1 , \$t0 , \$t1 # x = a + b

```
sub $v1, $v1, $t3 # x = x - d = a + b - d
addi $v1, $v1, 1 # x = x + 1 = a + b - d + 1

#y = b and c = 4
and $s0, $t1, $t2

#z = (a or b) and c = 4
or $s1, $t0, $t1 #z = a or b
and $s1, $s1, $t2 #z = z and c = (a or b) and c
```

**You were not born to fit into rooms built by
fearful minds. Build your own kingdom, and let
the world ask for directions**

